

Ejercicios de Práctica de Guías 3 y 4 de Haskell

Importante

Template de funciones a implementar [acá](#)

Lista de funciones permitidas [acá](#)

Ejemplo de hunit [acá](#)

Enunciado

Resolver los siguientes ejercicios cuyas especificaciones en lenguaje semiformal figuran a continuación. Deben ser implementadas en Haskell utilizando los tipos requeridos y solamente las funciones que se ven en la materia Introducción a la Programación / Algoritmos y Estructuras de Datos I (FCEyN-UBA).

1. Ejercicio 1 (2 puntos)

Implementar la función **todosMenores :: (Integer, Integer, Integer) -> Bool**

```
problema todosMenores (t: Z × Z × Z) : Bool
{
  requiere: {True}
  asegura: {res = true ⇔ (f(t0) > g(t0)) ∧
(f(t1) > g(t1)) ∧ (f(t2) > g(t2))}
}
```

```
problema f (n: Z) : Z {
  requiere: {True}
  asegura: {(n ≤ 7 → res = n2) ∧ (n > 7 → res
= 2n - 1)}
}
```

```

problema g (n: Z) : Z {
  requiere: {True}
  asegura: {Si n es un número par entonces
    res = n/2, en caso contrario, res = 3n+1}
}

```

Ejemplo: todosMenores (3,2,4) es Falso depende de f y g.

2. Ejercicio 2 (2 puntos)

Implementar una función **pitagoras :: Integer -> Integer -> Integer -> Integer**

Ejemplos:

pitagoras 3 4 5 → 20

pitagoras 3 4 2 → 6

```

problema pitagoras (m:  $\mathbb{N}_0$ , n:  $\mathbb{N}_0$ , r:  $\mathbb{N}_0$ ) :  $\mathbb{N}_0$ 
{
  requiere: {True}
  asegura: {res = cantidad de pares (p,q) con
     $0 \leq p \leq m$ ,  $0 \leq q \leq n$  tal que  $p^2 + q^2 \leq r^2$ }
}

```

3. Ejercicio 3 (2 puntos)

Implementar una función **mayorDigitoPar :: Integer -> Integer** según la siguiente especificación:

```

problema mayorDigitoPar (n:  $\mathbb{N}$ ) :  $\mathbb{N}$  {
  requiere: {True}
}

```

```

    asegura: {resultado es el mayor de los
    dígitos pares de n. Si n no tiene ningún
    dígito par, entonces resultado es -1.}
  }

```

Ejemplo: `mayorDigitoPar 2486` $\rightarrow$ 8 ;
`mayorDigitoPar 135` $\rightarrow$ -1

4. Ejercicio 4 (2 puntos)

Implementar la función `sonCoprimos :: Integer -> Integer -> Bool` que dados dos números naturales indica si no tienen algún divisor en común mayor estricto que 1.

```

problema sonCoprimos (a: ℕ, b: ℕ) : Bool {
  requiere: {True}
  asegura: {res = true  $\Leftrightarrow$  no existe  $d \in \mathbb{N}$ ,  $d > 1$  tal que d divide a a y d divide a b}
}

```

Ejemplos:

`sonCoprimos 8 15` $\rightarrow$ True

`sonCoprimos 14 21` $\rightarrow$ False (ambos divisibles por 7)

5. Ejercicio 5 (1 punto)

Conteste marcando la opción correcta asumiendo que α y β son dos fórmulas de la lógica proposicional tales que α es más fuerte que β , y que se tiene las siguientes especificaciones:

```

problema1 (...) {
  requiere: { $\alpha$ }
  asegura: { $\gamma$ }
}

```

```

problema2 (...) {
  requiere: { $\beta$ }
  asegura: { $\gamma$ }
}

```



Una correcta implementación de **problema1** va a contemplar al menos los mismos casos de entrada que una correcta implementación del **problema2**

☐

Una correcta implementación de **problema2** va a contemplar al menos los mismos casos de entrada que una correcta implementación del **problema1**

☐

No es posible afirmar ninguna de las opciones sin conocer en detalle ambas especificaciones

6. Ejercicio 6 (1 punto)

¿Cuándo se dice que una especificación está sobre-especificada?:

☐

Cuando se da una precondition más restrictiva de lo realmente necesario, o bien una postcondition más débil de la que se necesita.

☐

Cuando no hay precondiciones (o la precondition es True).

☐

Cuando se da una precondition más débil de lo realmente necesario, o bien una postcondition más restrictiva de la que se necesita.

Adjunta el archivo con tu solución:

Solo se puede adjuntar 1 archivo de extensión .hs. En caso de haber desarrollado tests propios, no deben ser entregados.

Seleccionar archivo a enviar:

Examinar...

No se ha seleccionado ningún archivo.

Enviar